

# Deliverable 1

## Structural Aliasing Analysis in Time-Series Forecasting Using Chronos

Matteo Boniotti, Gianluca Brignoli and Federico Sabbadini\*

July 30, 2026

This investigation formalizes the phenomenon of *structural aliasing* within patch-based time-series foundation models, focusing specifically on Amazon’s Chronos-Bolt framework.[1, 2] The project focuses on smart-grid load balancing.

### Domain Description

To ground this abstract architectural evaluation, the study utilizes a specific physical domain as an empirical reference layer. This reference setting models the solar energy produced by a photovoltaic array attached to an internal load, a local storage battery, and the national electricity grid. The explicit operational objective within this system is to evaluate production variations over short-term future horizons so that the system can dynamically modify the internal load, attach and detach the battery unit, with the aim of maintaining internal grid stability and enabling precise anomaly detection.

Within this infrastructure, a control AI agent is responsible for executing real-time tasks: dynamic load-balancing, battery dispatch, state-monitoring and active anomaly detection, and internal grid stability maintenance.

This analysis is directly intended for researchers, engineers and practitioners utilizing large language models within the Chronos family for time-series analysis, representation learning, and forecasting, as well as individuals working in the field of energy management who are interested in characterizing the fundamental limitations, structural boundaries, and latent capabilities of these architectures.

### Purpose and Scope

This project formalizes and empirically validates the phenomenon of *structural aliasing* in patch-based time-series foundation models, using Amazon Chronos-Bolt-Tiny as the target architecture.

Chronos-Bolt-Tiny operates with a fixed patch size of  $P = 16$ , stride of  $S = 16$  and context window of  $T = 2048$  tokens as default settings; its architecture comprises four encoder blocks followed by four decoder blocks terminating in a direct quantile projection head.[3, 4]

This study aims to provide a comprehensive understanding of the relationships between raw signal characteristics and the latent representations learned by the model, systematically identifying the critical

---

\*This work is licensed under the Creative Commons Attribution 4.0 International License. Concurrently, all associated source code, probing scripts, and software artifacts generated throughout the project lifecycle are released under the open-source MIT license.

Copyright for any third-party material included in this work remains with its respective owners and must be respected accordingly.

During the preparation of this work, the authors used AI-based tools, including GPT-5.5, Gemini 3.6 Flash, Sonnet 5, and Sonnet 4.6, to support language refinement and improve clarity. All generated or suggested content was carefully reviewed, revised where necessary, and validated by the authors, who assume full responsibility for the final content of this work.

links between sampling frequency ( $f_s$ ), patch size ( $P$ ), and stride ( $S$ ) in order to propose actionable solutions to mitigate the phenomenon.

The core objective is to challenge the baseline assumption that pre-trained temporal foundation models inherently preserve frequency attributes when processing Nyquist-compliant signals.

## Mathematical Formulation

The project provides a formal domain description through OWL2 ontology representation of: the physical system (photovoltaic array, battery storage, national grid, internal load), the signal (frequency, amplitude, timespan, phase) and its patched representation (patch size, stride, sampling frequency). See Appendix A for a formal description of the ontology.

The investigation focuses on the formal definition of the phenomenon of structural aliasing, starting from the tokenization operator and describing phase-locking conditions. The project then formalizes the relationship between the signal frequency, patch size, stride, and sampling frequency, providing a mathematical framework to analyze the conditions under which structural aliasing occurs.

Let  $\mathbf{x} = \{x[n]\}_{n \in \mathbb{Z}}$  be a discrete-time signal representing the input time series. We define the patch-based tokenization operator  $\mathcal{T}_{P,S}$  with patch length  $P \in \mathbb{N}^+$  and stride  $S \in \mathbb{N}^+$  as a mapping that transforms the signal  $\mathbf{x}$  into a sequence of vector-valued tokens  $\mathbf{T} = \{\mathbf{t}_k\}_{k \in \mathbb{Z}}$ , where each token  $\mathbf{t}_k \in \mathbb{R}^P$ . The  $k$ -th token is defined as

$$\mathbf{t}_k = \mathcal{T}_{P,S}(\mathbf{x})[k] = \begin{bmatrix} x[kS] \\ x[kS + 1] \\ \vdots \\ x[kS + P - 1] \end{bmatrix}. \quad (1)$$

Considering a continuous-time periodic signal sampled uniformly at sampling frequency  $f_s$ , yielding the discrete-time sequence  $x[n]$ . We isolate a fundamental harmonic component as

$$x[n] = \sin\left(2\pi \frac{f}{f_s} n + \phi\right), \quad (2)$$

where  $f$  denotes the signal frequency satisfying the Nyquist criterion  $f < \frac{f_s}{2}$ , and  $\phi \in [0, 2\pi)$  is an arbitrary phase offset.

Phase-locking occurs when the structural phase profile of the signal repeats exactly under a translational shift equal to the stride  $S$ . Evaluating a stride translation yields

$$x[n + S] = \sin\left(2\pi \frac{f}{f_s} (n + S) + \phi\right) = \sin\left(2\pi \frac{f}{f_s} n + 2\pi \frac{fS}{f_s} + \phi\right). \quad (3)$$

For exact alignment between the signal and the tokenization operator, the phase accumulation over one stride must correspond to an integer number of  $2\pi$ -periods

$$2\pi \frac{fS}{f_s} = 2\pi c, \quad c \in \mathbb{N}^+. \quad (4)$$

As a result, the signal exhibits exact stride-period translational invariance

$$x[n + S] = \sin\left(2\pi \frac{f}{f_s} n + 2\pi c + \phi\right) = \sin\left(2\pi \frac{f}{f_s} n + \phi\right) = x[n]. \quad (5)$$

Solving Equation (4) for  $f$  yields the phase-locking frequency class  $\mathcal{F}_{\text{lock}}$ , which is defined as the set of frequencies producing stride-period self-repetition:

$$\mathcal{F}_{\text{lock}} = \left\{ f \in \mathbb{R}^+ \mid f = c \frac{f_s}{S}, c \in \mathbb{N}^+, f < \frac{f_s}{2} \right\}. \quad (6)$$

The derivation above shows that a stride translation of length  $S$  leaves a single stride-locked sinusoid unchanged,  $x_f[n + S] = x_f[n]$  for  $f \in \mathcal{F}_{\text{lock}}$ : this repetition of consecutive patches of one signal is not an evidence that two distinct signals become indistinguishable to the model.

Structural aliasing is treated as an emergent, falsifiable property of the trained representations: two frequency-matched but distinct signals  $x, y$  structurally alias at layer  $l$  if:

$$\|R_l(x) - R_l(y)\|_2 \leq \varepsilon \quad (7)$$

where  $R_l$  is the representation at layer  $l$ ,  $\|\cdot\|_2$  is the Euclidean norm, and  $\varepsilon$  is a pre-registered threshold.

## Aliasing implications

The above derivation reframes  $\mathcal{F}_{\text{lock}}$  as the candidate class where such collisions are hypothesised to occur. In particular, the project will test the following hypotheses.

**H1:** if  $f \in [f_k - \varepsilon, f_k + \varepsilon]$ , where  $f_k \in \mathcal{F}_{\text{lock}}$ , the model recovers less information, we expect: higher MDL codelength and lower forecast amplitude recovery than at fixed controls  $f_k \pm \delta$ ,  $\delta \gg \varepsilon$ . Absent or non-localized loss refutes H1.

**H2:** the magnitude of degradation at a locked frequency  $f_k \in \mathcal{F}_{\text{lock}}$  is a property of the stride/patch alignment, not of the signal's own phase: across the  $S_{f_k}$  phase offsets already sampled at  $f_k$ , the accuracy drop relative to the  $f_k \pm \delta$  control is expected to stay approximately constant. A drop whose size varies systematically with phase, rather than only with frequency, refutes H2.

**H3:** when  $S$  is varied while  $P$  is fixed, the locations of any local degradation will move proportionally to  $1/S$ , tracking  $\mathcal{F}_{\text{lock}}(S)$ . An effect that remains tied to  $P$  rather than  $S$  will refute the proposed stride-lock mechanism and H3.

## Data and Evaluation

Two sets of synthetic signals are used to evaluate the presence of structural aliasing in the new models. These include:

- **Sine Wave Signals:** simple sine waves with known frequency systematically varied between 2 Hz and 250 Hz with 1 Hz increments, to assess the model's ability to capture and represent periodic components across a range of frequencies.
- **Time Series Mixup Signals:** 100 signals generated by the light version of TSMixup[1] procedure, mixing multiple sine waves of different frequencies; to evaluate the model's capacity to disentangle and represent complex frequency interactions.

This set of 349 test signals is used for all the re-trained models, to evaluate the impact of stride and patch size on the model's ability to capture and represent frequency information.

Optionally, if previous analyses show evidence of aliasing, the project may analyze test signals from a third set, including 100 signals generated using the KernelSynth[1] tool, to test whether the phenomenon persists in more complex signals. The project will then focus on the analysis of real-world data, through the use of the `Dataset-SolarTechLab.csv` dataset, which contains a one-minute-resolution multivariate telemetry stream of a photovoltaic array[5].

Also, those tests are undertaken for all the re-trained models, to evaluate the impact of stride and patch size on the model's ability to capture and represent frequency information in more complex signals. During the analysis a SEED is set to ensure reproducibility of the results.

| Run | Patch ( $P$ ) | Stride ( $S$ ) | overlap | #patch @2048 | steps             | time  |
|-----|---------------|----------------|---------|--------------|-------------------|-------|
| 1   | 16            | 4              | 0.75    | 509          | 100k              | 4.1 h |
| 2   | 16            | 8              | 0.50    | 255          | 100k              | 2.6 h |
| 3   | 16            | 12             | 0.25    | 170          | 100k              | 2.8 h |
| 4   | 16            | 16             | 0.00    | 128          | 200k <sup>†</sup> | —     |
| 5   | 8             | 8              | 0.00    | 256          | 100k              | 2.7 h |
| 6   | 24            | 24             | 0.00    | 86           | 100k              | 2.8 h |

**Table 1:** Patch/stride configurations retrained from scratch (seed 42); time is the wall-clock on the RTX 5060 Laptop GPU. <sup>†</sup> The  $P=16, S=16$  point is the official Amazon checkpoint (200k steps), used as baseline and not retrained.

## Methodology

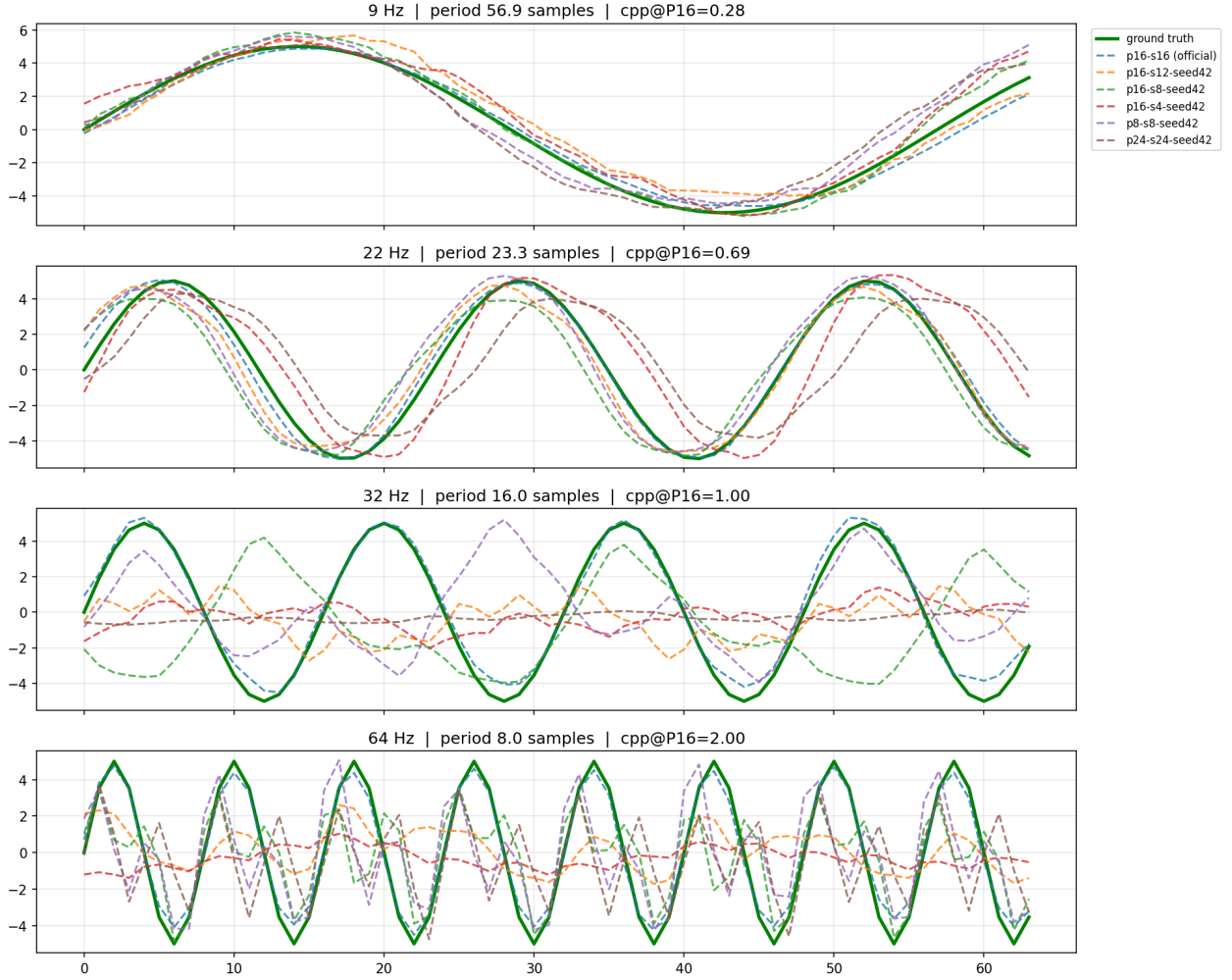
Initially, the Chronos Bolt-tiny model available on the Hugging Face Hub is used to evaluate the presence of structural aliasing in the default configuration, with patch size  $P = 16$  and stride  $S = 16$ . To evaluate the presence and impact of structural aliasing, the project will analyze a set of synthetic signals with varying frequency characteristics. Then the Chronos Bolt-tiny model is tested on simple sine waves with known frequency systematically varied between 2 Hz and 250 Hz with 1 Hz increments, those tests are used to evaluate the presence of structural aliasing.

Since any change to  $P$  or  $S$  requires retraining, we retrain a restricted grid around the default  $P = 16, S = 16$  baseline (see Table 1): three reduced strides at fixed  $P = 16$  ( $S = 12, 8, 4$ ) isolate the effect of patch overlap, while two contiguous patch sizes ( $P = 8, 24$ ) isolate that of patch width. The extreme strides  $S = 1$  and  $S = 15$  at  $P = 16$  remain as possible extensions, probing the largest and smallest overlap.

| Parameter                                   | Adopted                      | Chronos official | Max / constraint  |
|---------------------------------------------|------------------------------|------------------|-------------------|
| <i>Architecture (fixed, from Bolt-tiny)</i> |                              |                  |                   |
| context_length                              | 2048                         | 2048             | Bolt-tiny         |
| prediction_length                           | 64                           | 64               | Bolt-tiny         |
| quantile levels                             | 9                            | 9                | Bolt-tiny         |
| <i>Optimisation</i>                         |                              |                  |                   |
| batch_size                                  | 32                           | 32               | VRAM 8 GB         |
| max_steps                                   | 100 000                      | 200 000          | compute budget    |
| learning_rate                               | $10^{-3}$                    | $10^{-3}$        | —                 |
| scheduler / warmup                          | linear / 0                   | linear / 0       | —                 |
| weight_decay                                | 0.0                          | 0.0              | —                 |
| grad_clip_norm                              | 1.0                          | 1.0              | —                 |
| optimiser / precision                       | AdamW (fused) / fp32+TF32    | —                | —                 |
| <i>Data pipeline</i>                        |                              |                  |                   |
| corpora (streaming)                         | TSMixup 10M + KernelSynth 1M |                  | official          |
| mixing ratio                                | 9:1                          | 9:1              | —                 |
| shuffle_buffer                              | 10 000                       | 100 000          | streaming timeout |
| min_past / max_missing / drop_prob          | 60 / 0.9 / 0.2               | —                | —                 |

**Table 2:** Training hyperparameters: adopted value, official Chronos recipe (*chronos-t5-tiny*), and usable maximum / constraint. Only ( $P, S$ ), the step budget (100k vs. 200k) and the shuffle buffer (10k vs. 100k) depart from the official values, to fit the single-GPU budget.

Each ( $P, S$ ) configuration is retrained from scratch: changing  $P$  resizes the first patch-embedding block, so weights are not transferable. Loading only the **chronos-bolt-tiny** architecture, the loop reproduces the official recipe (fused AdamW, linear LR  $10^{-3}$ , **fp32+TF32**, masked quantile loss) on the two official corpora: TSMixup (10M) and KernelSynth (1M) streamed at the 9:1 ratio[6]. Every hyperparameter stays at its official



**Figure 1:** Forecast quality of the retrained models. Per frequency (rows), each 64-step forecast (dashed) vs. ground truth (green); titles give cycles-per-patch at  $P = 16$  (cpp). The real model performances are better than the retrained, due to the larger training budget and more extensive corpora.

value (see Table 2); only  $(P, S)$  varies, and a uniform 100k-step budget (vs. 200k) is used since the variants are compared only against each other.

Training ran on one Alienware 16 Aurora (NVIDIA RTX 5060 Laptop GPU, 8 GB VRAM; 32 GB RAM),  $\approx 15$  h in total (2.6–4.1 h per model).

Forecasting quality of the retrained models is shown in Figure 1. If necessary, the project may extend the training budget to the full parameters.

## Probing Methodology

Following MDL probing [7], the 256-dimensional [REG] representation is captured after each encoder block and each decoder block, plus two points at the output stage: the pre-projection 256-d [REG] input, and the projected quantile output (Pagani et al.’s  $h_4$  [8]), where the compression drop reported in the source paper is concentrated. Probing this single per-block vector, rather than the full flattened patch sequence, avoids a dimensionality mismatch across stages and patch/stride variants.

All probe points are compared on the same footing via a prequential-MDL codelength  $L(D)$ , reported

as compression  $SV = 1 - L(D)/L_{\text{uniform}}(D)$ , computed independently per probe point for each of the seven hierarchical band-classification tasks [8]. Each value is benchmarked against a shuffled-label control ( $SV \leq 0$  confirms the probe is not memorising noise) and a random-initialisation control (identical untrained architecture, isolating learned information from architectural artefacts).

Independently, a dense per-frequency classification-accuracy profile is obtained at the projected output by sweeping the same band-classification probe across the operational spectrum, with non-redundant phase sampling  $S_f = \min\{f_s / \gcd(f, f_s) - 1, N\}$  replacing independently drawn random phases.

## Bayesian Analysis

We employ a two-part Bayesian framework to quantify both relative local degradation and absolute performance losses across hypotheses  $H1$ – $H3$ .

**Local Contrast Analysis:** to test localized recovery drop at stride-lock for each retrained model configuration  $(P, S)$  and training seed, signals at exact  $f_k \in \mathcal{F}_{\text{lock}}$  are paired with control signals at offset frequencies  $f_k \pm 0.25f_s/S$ , keeping phase, baseline amplitude, noise level, and background process parameters identical. Letting  $R$  denote the forecast amplitude recovery ratio, we define the paired logarithmic local contrast  $d$  as:

$$d = \log(R_k + 0.01) - \frac{\log(R_- + 0.01) + \log(R_+ + 0.01)}{2} \quad (8)$$

where  $R_k$  is the recovery at  $f_k$ , and  $R_{\pm}$  are recoveries at the control frequencies  $f_k \pm 0.25f_s/S$ . A contrast value of  $d < 0$  directly indicates localized forecast attenuation induced by phase-locking.

Contrasts are modeled using a heavy-tailed Student- $t_4(\mu_i, \sigma^2)$  likelihood. The population phase-lock effect  $\bar{\beta}$  receives a weakly informative prior  $\bar{\beta} \sim \text{Student-}t_4(0, 0.5)$ , centered at zero due to the absence of prior quantitative estimates in the literature.

**Absolute Performance Analysis:** to evaluate raw non-negative metric specifically prequential MDL codelength  $L(D)$ , we use a Bayesian Gamma regression model with a log link:

$$y_i \sim \text{Gamma}(\text{shape} = k, \text{mean} = \mu_i) = \alpha_0 + \theta_{\text{lock}} \cdot \text{IsLocked} \quad (9)$$

where  $\text{IsLocked} \in \{0, 1\}$  flags phase-lock conditions, and  $\theta_{\text{lock}} \sim \mathcal{N}(0, 0.5)$  captures the relative codelength expansion.

The Gamma shape parameter receives a weakly informative prior,  $k \sim \text{Gamma}(2, 0.1)$ , allowing the model to adaptively capture metric dispersion.

**Inference:** planned posterior inference will evaluate 95% credible intervals and the posterior probability of at least 20% attenuation.

## Security Implications and Risk Assessment

Finally, the project will include a formal risk assessment grounded in the NIST AI Risk Management Framework (AI RMF) [9] to evaluate structural aliasing as a systematic vulnerability within cyber-physical energy systems. This analysis will outline plausible threat models and attack surfaces, possible mitigation strategies (including stride reduction), and an evaluation of whether the proposed mitigation strategies effectively reduce residual exposure for the automated control AI agent.

The risk assessment will be structured around the following key points: context and attack surface, threat modeling, impact, mitigation and residual exposure.

## Significance and Impact

This subject is relevant because it explores an unexamined systemic vulnerability native to patch-based temporal foundation architectures. By systematically characterizing the conditions under which structural aliasing emerges, this work will provide critical insights into the fundamental limitations of these models, directly informing the design of more robust architectures and training paradigms.

For professionals operating within this domain, the analysis could expose how algorithmic data-deletion anomalies can propagate through automated distribution pipelines. By evaluating how hidden token-space representation collapse can deceive a load-balancing AI agent into executing misaligned control choices, this work could provide insights needed for high-reliability, fault-tolerant smart-grid deployments.

The impact of this project could extend beyond theoretical understanding, as it has direct implications for the reliability and safety of real-world applications that depend on accurate time-series forecasting and representation learning, particularly in high-stakes domains such as energy management.

## Appendix A: OWL2 Ontology

The core ontology architecture bridges the gap between cyber-physical energy systems and deep learning signal processing domain knowledge. It categorizes the application setting into five main conceptual pillars rooted under the ontological hierarchy:

- **Physical Subsystem (pv:PhysicalComponent):** Represents the hardware infrastructure of the micro-grid, including the solar array, electrochemical storage, consumer loads, and the macro grid tie.
- **Cognitive Layer (pv:ControlAgent):** The artificial intelligence system responsible for autonomous grid optimization, load balancing, dispatch scheduling, and proactive vulnerability mitigation.
- **Signal Processing Layer (pv:Signal, pv:PatchedSignal):** Tracks the raw telemetry inputs (both time-domain features and frequency-domain components) along with their tokenized representation configurations (patch dimensions, stride values) tailored for the Chronos foundation architecture.
- **Environmental Tracking (pv:WeatherObservation):** Captures concurrent meteorological snapshots (air temperature, wind vectors) influencing structural physics and electrical efficiency.
- **Inferred Operational States:** Dynamic metadata classified into generation levels (pv:GenerationState), systemic disruptions (pv:AnomalyState), control directives (pv:AgentAction), and signal degradation hazards (pv:AliasingEvent).

### Generation States

A formal description of the state of the photovoltaic array is provided through the following axioms, which define the generation state based on the measured tilted irradiance in  $\text{W/m}^2$ :

$$\begin{aligned} \text{PhotovoltaicArray}(?pv) \wedge \text{measures}(?pv, ?s) \wedge \text{tiltedIrradianceWm2}(?s, ?g) \wedge (?g \geq 800.0) \\ \longrightarrow \text{hasGenerationState}(?pv, \text{ClearSky}) \end{aligned} \quad (10)$$

$$\begin{aligned} \text{PhotovoltaicArray}(?pv) \wedge \text{measures}(?pv, ?s) \wedge \text{tiltedIrradianceWm2}(?s, ?g) \\ \wedge (?g \geq 150.0) \wedge (?g < 800.0) \\ \longrightarrow \text{hasGenerationState}(?pv, \text{PartialCloud}) \end{aligned} \quad (11)$$

$$\begin{aligned} \text{PhotovoltaicArray}(?pv) \wedge \text{measures}(?pv, ?s) \wedge \text{tiltedIrradianceWm2}(?s, ?g) \\ \wedge (?g > 0.0) \wedge (?g < 150.0) \\ \longrightarrow \text{hasGenerationState}(?pv, \text{Overcast}) \end{aligned} \quad (12)$$

$$\begin{aligned} \text{PhotovoltaicArray}(?pv) \wedge \text{measures}(?pv, ?s) \wedge \text{tiltedIrradianceWm2}(?s, ?g) \wedge (?g \leq 0.0) \\ \longrightarrow \text{hasGenerationState}(?pv, \text{Nighttime}) \end{aligned} \quad (13)$$

### Anomaly States

Formal description of the anomaly states which could be detected in the photovoltaic array, based on the measured irradiance and the measured power.

### Aliasing Event

Formal description of the aliasing event, detected when the frequency of the signal is part of the frequency class  $\mathcal{F}_{\text{lock}}$ .

### Agent Actions

Formal description of the action taken by the control agent, based on the current state and aliasing event.



## References

- [1] Abdul Fatir Ansari et al. *Chronos: Learning the Language of Time Series*. Nov. 4, 2024. DOI: 10.48550/arXiv.2403.07815. arXiv: 2403.07815 [cs.LG]. URL: <http://arxiv.org/abs/2403.07815> (visited on 05/13/2026). Pre-published.
- [2] Abdul Fatir Ansari et al. *Fast and Accurate Zero-Shot Forecasting with Chronos-Bolt and AutoGluon / Artificial Intelligence*. Dec. 2, 2024. URL: <https://aws.amazon.com/blogs/machine-learning/fast-and-accurate-zero-shot-forecasting-with-chronos-bolt-and-autogluon/> (visited on 05/13/2026).
- [3] *Amazon/Chronos-Bolt-Tiny · Hugging Face*. Dec. 4, 2025. URL: <https://huggingface.co/amazon/chronos-bolt-tiny> (visited on 07/25/2026).
- [4] *ChronosBoltPipeline | Amazon-Science/Chronos-Forecasting*. DeepWiki. URL: <https://deepwiki.com/amazon-science/chronos-forecasting/2.4-chronosboltpipeline> (visited on 05/13/2026).
- [5] Sonia Leva et al. *Photovoltaic Power and Weather Parameters*. IEEE DataPort, Sept. 23, 2020. DOI: 10.21227/42V0-JZ14. URL: <https://ieee-dataport.org/open-access/photovoltaic-power-and-weather-parameters> (visited on 06/01/2026).
- [6] Abdul Fatir Ansari et al. *Chronos: Learning the Language of Time Series [SW]*. 2024. arXiv: 2403.07815. URL: <https://github.com/amazon-science/chronos-forecasting> (visited on 07/06/2026).
- [7] Elena Voita and Ivan Titov. “Information-Theoretic Probing with Minimum Description Length”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. EMNLP 2020. Ed. by Bonnie Webber et al. Online: Association for Computational Linguistics, Nov. 2020, pp. 183–196. DOI: 10.18653/v1/2020.emnlp-main.14. URL: <https://aclanthology.org/2020.emnlp-main.14/> (visited on 05/13/2026).
- [8] Alessandro Pagani et al. *Preliminary Insights in Chronos Frequency Data Understanding and Reconstruction*. May 7, 2026. DOI: 10.48550/arXiv.2605.06361. arXiv: 2605.06361 [cs.LG]. URL: <http://arxiv.org/abs/2605.06361> (visited on 05/13/2026). Pre-published.
- [9] Elham Tabassi. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1. Gaithersburg, MD: National Institute of Standards and Technology (U.S.), Jan. 26, 2023, NIST AI 100-1. DOI: 10.6028/NIST.AI.100-1. URL: <http://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf> (visited on 06/01/2026).